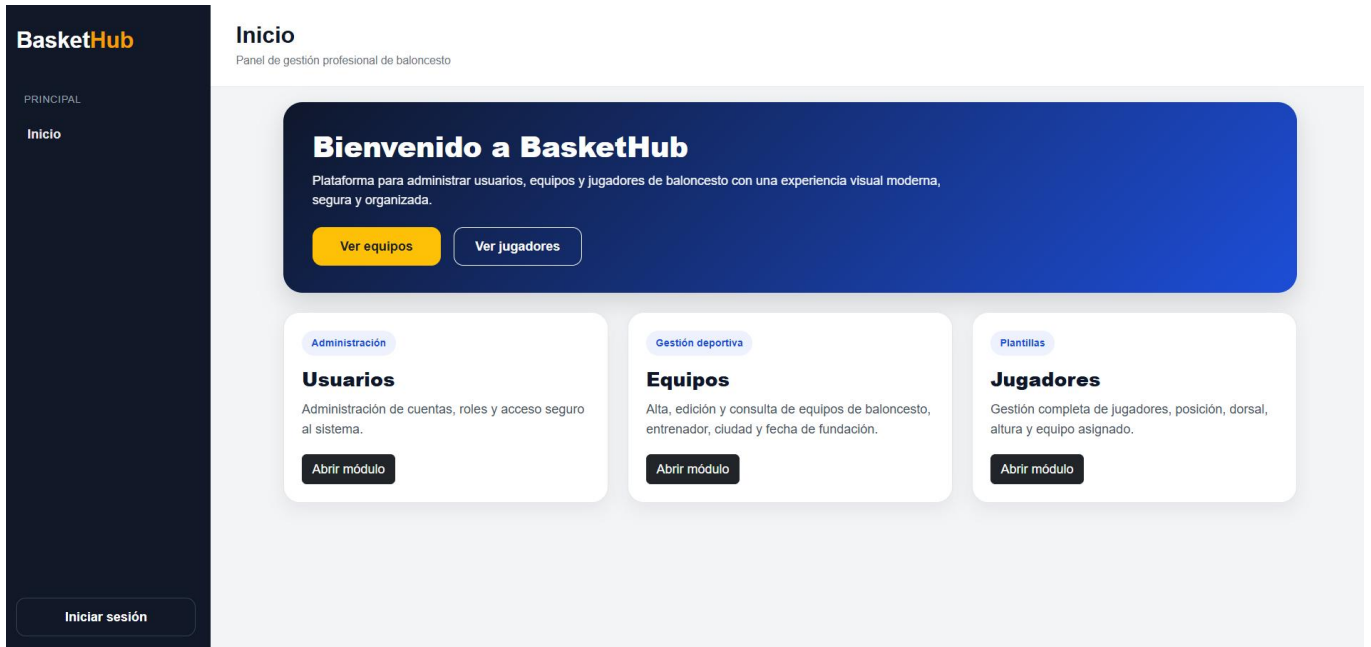


BasketHub

Mario Carcedo Martínez

Aplicación web desarrollada con Spring Boot para la gestión de usuarios, equipos y jugadores de baloncesto. El proyecto parte de la práctica anterior y amplía la interfaz con un diseño más profesional, seguridad por roles, despliegue con Docker e integración básica de Google Maps y SonarQube.



1. Objetivo y alcance

El objetivo del proyecto ha sido construir un frontend sofisticado y visualmente agradable, con temática de baloncesto, que permitiera gestionar información de usuarios, equipos y jugadores desde una arquitectura web en capas. La aplicación cumple con los requisitos obligatorios del taller y añade varias mejoras opcionales.

Arquitectura utilizada

- Cliente web: navegador con vistas HTML renderizadas por Thymeleaf.
- Backend: Spring Boot con controladores MVC, servicios y repositorios.
- Persistencia: JPA + Hibernate sobre MySQL.
- Seguridad: Spring Security con login personalizado, logout y control de acceso por roles.
- Interfaz: Bootstrap, layout tipo dashboard con sidebar lateral.
- Despliegue: Docker / Docker Compose.

Roles y permisos

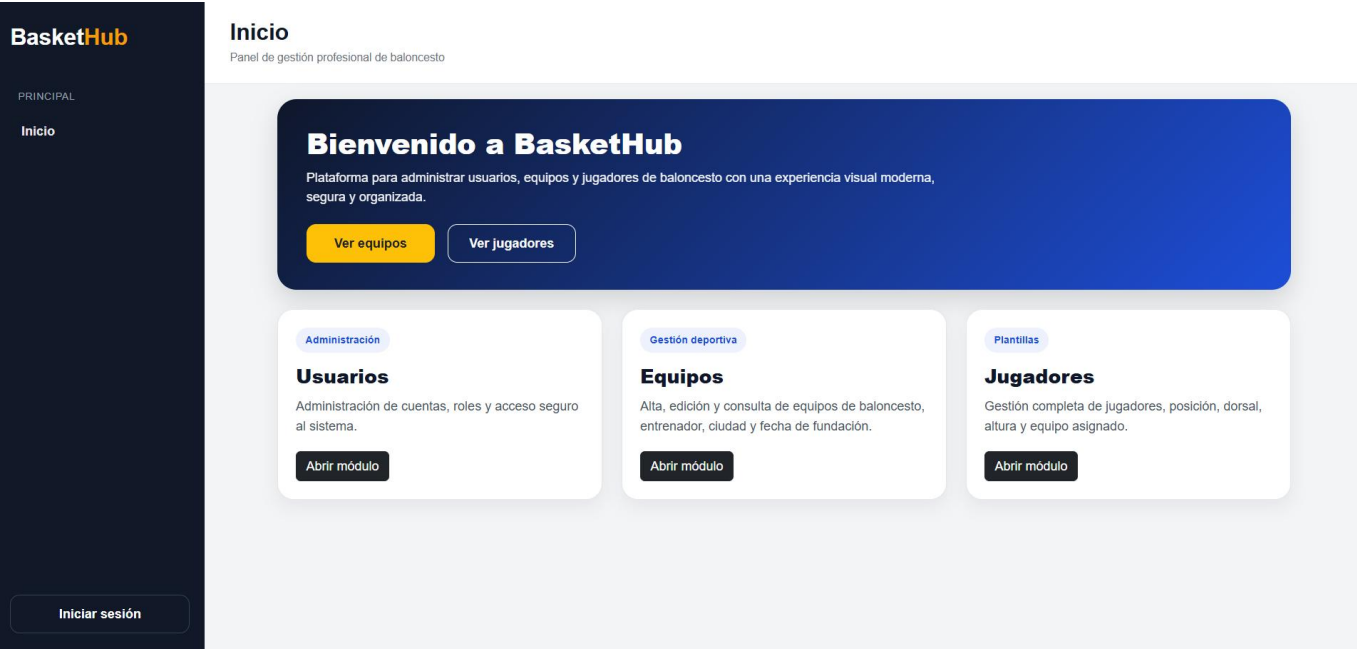
Rol	Permisos
ADMIN	Acceso completo. Gestiona usuarios, equipos y jugadores.
MANAGER	Gestiona equipos y jugadores, sin acceso a mantenimiento de usuarios.
PLAYER	Acceso de solo lectura a equipos, jugadores y mapa.

Funcionalidades implementadas

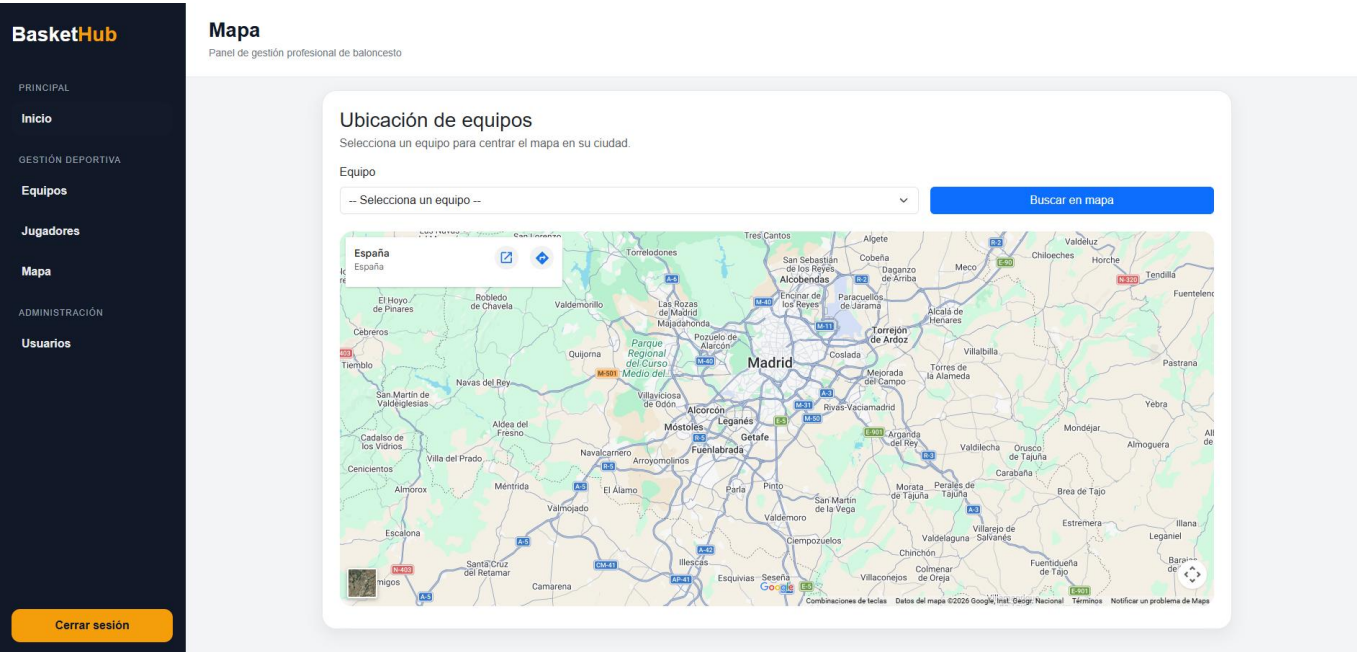
- Página principal con navegación lateral y acceso rápido a módulos.
- Pantalla de login personalizada y cierre de sesión seguro.
- Pantalla de mantenimiento de usuarios (alta, edición, borrado y consulta).
- CRUD completo de equipos.
- CRUD completo de jugadores.
- Pantalla 403 de acceso denegado.
- Protección para evitar borrar el último usuario administrador.
- Integración básica de Google Maps para localizar equipos por ciudad.
- Análisis estático del código con SonarQube.

2. Evidencias visuales

Dashboard principal



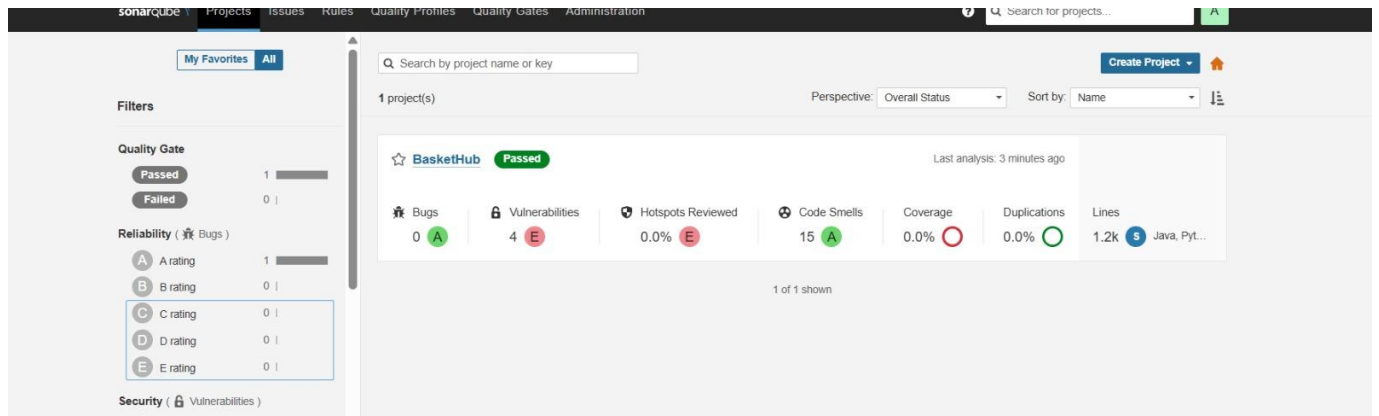
Integración de mapas



3. Calidad y despliegue

Análisis de calidad con SonarQube

Se ha realizado un análisis estático del proyecto con SonarQube para revisar la calidad del código, vulnerabilidades y hotspots de seguridad. En una primera ejecución se detectaron incidencias relacionadas con credenciales hardcoded y puntos de seguridad pendientes de revisión. Posteriormente, estas incidencias fueron corregidas en el commit 78b46bce5751abb7985cf391fd3ccf3a96b28c41, dejando el proyecto sin bugs ni vulnerabilidades activas en el análisis.



El estado del *Quality Gate* puede seguir apareciendo como Failed en algunos análisis debidos exclusivamente a la cobertura de tests, ya que en esta iteración no se han desarrollado pruebas automáticas y la cobertura del código nuevo es del 0.0%. Por tanto, el fallo del *Quality Gate* no se debe a vulnerabilidades de seguridad, sino a la ausencia de tests de cobertura.

